Coordinator Agent Architecture Walkthrough

Overview

The Coordinator Agent has been successfully deployed, acting as the intelligent orchestrator for the entire CropGuardian backend. By wrapping the pipeline (Detection Severity Advisory) into a unified Facade pattern, we've drastically simplified the Streamlit UI and prepared the system for a future headless API server rollout (e.g., FastAPI).

Changes Made

- **Coordinator Agent Module**:
  - Located at [coordinator_agent.py] (file:///d:/agricare/agents/coordinator_agent/coordinator_agent.py).
  - Internally manages instantiations of the `DiseaseDetectionAgent`, `SeverityAgent`, and `AdvisoryAgent`.
  - `process_image()` interface yields a robust structured JSON response including a `status`, execution time, and `agent_trace`.
  - Gracefully handles **partial successes**: If the Gemini API times out (and the fallback also fails), the Coordinator will log a warning but still successfully return the valid Disease and Severity results to the user.
- **Streamlit Refactoring**:
  - The [1_Disease_Detection.py] (file:///d:/agricare/app/pages/1_Disease_Detection.py) page no longer calls multiple agents sequentially. It exclusively utilizes `CoordinatorAgent.process_image()`.
  - Added a *Pipeline Diagnostics* footer to the disease report which exposes the trace, execution time, and status.
  - Preserved the separation of the `FeedbackAgent`, keeping asynchronous feedback out of the synchronous inference pipeline.
- **Unit Testing**:

- Handled complete mock integration tests in [test_coordinator_agent.py] (file:///d:/agricare/tests/test_coordinator_agent.py) covering success paths, partial-success paths (simulating Advisory failure), and invalid image inputs.

Final Project Structure

The CropGuardian AI multi-agent MVP is now fully realized and cleanly structured:

```
d:/agricare/
    agents/                    #  Multi-Agent Architecture
        advisory_agent/          # Gemini LLM logic + fallback
        coordinator_agent/       # Unified API orchestration
        disease_detection_agent/   # Core MobileNetV2 inference
        feedback_agent/          # Continuous learning & DB writes
        severity_agent/          # Rule-based logic evaluation
    app/                       #  Streamlit MVP Frontend
        main.py                  # Navigation routing
        pages/
            1_Disease_Detection.py  # Calls Coordinator + Feedback
            2_Feedback.py        # Manual feedback upload
            3_History.py         # Prediction logs
            4_Model_Insights.py    # Metrics & KPIs
    data/                      #  Local Data Stores
        dummy_test/              # Testing images
        feedback/                # User-corrected images & DB
        history/                 # Prediction audit trails
        retraining_dataset/        # Exported feedback for training
    models/                    #  Pretrained Artifacts
        class_names.npy
        crop_disease_mobilenetv2.keras
    reports/                   #  Output from Evaluation Pipeline
    src/                       #  Core Logic
        evaluation/              # Evaluation Pipeline Tooling
    tests/                     #  Comprehensive Unit Tests
        test_advisory_agent.py
        test_coordinator_agent.py
        test_disease_detection_agent.py
        test_evaluation_pipeline.py
        test_feedback_agent.py
```

```
        test_severity_agent.py
    requirements.txt
```

Validation Results

- Ran `venv\Scripts\python -m unittest tests/test_coordinator_agent.py`.
- `Ran 3 tests in 2.136s OK`.
- The Coordinator successfully stitched together the outputs, handled simulated API exceptions gracefully, and generated correct execution traces.

The MVP is feature-complete and beautifully architected!